K.R. MANGALAM UNIVERSITY

Database Management Lab File

(ENCS205)

Bachelor of
Technology in
Computer Science and Engineering

B. Tech CSE (Core)

4 Semester

School of Engineering & Technology

K. R. MANGALAM UNIVERSITY Sohna, Haryana
122103, India Jan-2026

Name – Pooja Aggarwal

Roll No – 2401010240

Submitted To:

Mansi Kajal

EXPERIMENT-3

Database Schema for a customer-sale scenario
Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)
For the above schema, perform the following-

1. Create the tables with the appropriate integrity constraints.

```sql
CREATE DATABASE customer_sale_db;
USE customer_sale_db;

```

26	14:17:50	CREATE DATABASE customer_sale_db	1 row(s) affected
27	14:17:50	USE customer_sale_db	0 row(s) affected

```sql
USE customer_sale_db;
-- Customer table
CREATE TABLE Customer(
    cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50) NOT NULL
);

-- Item table
CREATE TABLE Item(
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50) NOT NULL,
    price INT NOT NULL CHECK (price > 0)
);

-- Sale table
CREATE TABLE Sale(
    bill_no INT,
    bill_date DATE NOT NULL,
    cust_id INT,
    item_id INT,
    qty_sold INT NOT NULL CHECK (qty_sold > 0),

    PRIMARY KEY (bill_no, item_id),

    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,

    FOREIGN KEY (item_id) REFERENCES Item(item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

27	14:17:50	USE customer_sale_db	0 row(s) affected
28	14:20:24	USE customer_sale_db	0 row(s) affected
29	14:20:24	CREATE TABLE Customer(cust_id INT PRIMARY KEY, cust_name VARCHAR(50) NOT NULL)	0 row(s) affected
30	14:20:24	CREATE TABLE Item(item_id INT PRIMARY KEY, item_name VARCHAR(50) NOT NULL, price INT ...	0 row(s) affected
31	14:20:24	CREATE TABLE Sale(bill_no INT, bill_date DATE NOT NULL, cust_id INT, item_id INT, qty_sold...	0 row(s) affected

2. Insert around 10 records in each of the tables.

```
INSERT INTO Customer VALUES
(1,'Vanshika Dubey'),
(2,'Riya'),
(3,'Karan'),
(4,'Sneha'),
(5,'Rahul'),
(6,'Priya'),
(7,'Arjun'),
(8,'Neha'),
(9,'Vikas'),
(10,'Anjali');
```

33	14:23:41	INSERT INTO Customer VALUES (1,'Vanshika Dubey'), (2,'Riya'), (3,'Karan'), (4,'Sneha'), (5,'Rahul'), (6,'Priya'),...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

```
INSERT INTO Item VALUES
(101,'Pen',10),
(102,'Notebook',50),
(103,'Bag',500),
(104,'Bottle',150),
(105,'Shoes',1200),
(106,'Watch',2000),
(107,'Calculator',350),
(108,'Mouse',300),
(109,'Keyboard',600),
(110,'Headphones',800);
INSERT INTO Sale VALUES
(1,CURDATE(),1,101,5),
(1,CURDATE(),1,102,2),
(2,CURDATE(),2,103,1),
(3,CURDATE(),3,104,3),
(4,CURDATE(),4,105,1),
(5,CURDATE(),5,106,2),
(6,CURDATE()-1,6,107,1),
(7,CURDATE()-2,7,108,4),
(8,CURDATE()-3,8,109,2),
(9,CURDATE()-4,9,110,1);
```

42	14:31:19	INSERT INTO Item VALUES (101,'Pen',10), (102,'Notebook',50), (103,'Bag',500), (104,'Bottle',150), (105,'Sho...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
43	14:31:19	INSERT INTO Sale VALUES (1,CURDATE(),1,101,5), (1,CURDATE(),1,102,2), (2,CURDATE(),2,103,1), (3,CU...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0

3. List all the bills for the current date with the customer names and item numbers.

```sql
USE customer_sale_db;
SELECT s.bill_no, s.bill_date, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_name	item_id
1	2026-02-28	Vanshika Dubey	101
1	2026-02-28	Vanshika Dubey	102
2	2026-02-28	Riya	103
3	2026-02-28	Karan	104
4	2026-02-28	Sneha	105
5	2026-02-28	Rahul	106

Result 21

Output

Action Output

#	Time	Action	Message
44	14:34:01	USE customer_sale_db	0 row(s) affected
45	14:34:01	SELECT s.bill_no, s.bill_date, c.cust_name, s.item_id FROM Sale s JOIN Customer c ON s.cust_id = c.cust_id ...	6 row(s) returned

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```sql
USE customer_sale_db;
SELECT
    s.bill_no,
    i.item_name,
    s.qty_sold,
    i.price,
    (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	item_name	qty_sold	price	final_amount
1	Pen	5	10	50
1	Notebook	2	50	100
2	Bag	1	500	500
3	Bottle	3	150	450
4	Shoes	1	1200	1200
5	Watch	2	2000	4000
6	Calculator	1	350	350
7	Mouse	4	300	1200
8	Keyboard	2	600	1200
9	Headphones	1	800	800

Result 22

Output

Action Output

#	Time	Action	Message
46	14:36:38	USE customer_sale_db	0 row(s) affected
47	14:36:38	SELECT s.bill_no, i.item_name, s.qty_sold, i.price, (s.qty_sold * i.price) AS final_amount FROM Sa...	10 row(s) returned

5. List the details of the customer who have bought a product which has a price>200.

```
USE customer_sale_db;
SELECT DISTINCT c.*
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name
2	Riya
4	Sneha
5	Rahul
6	Priya
7	Arjun
8	Neha
9	Vikas

Result 23

Output

Action Output

#	Time	Action	Message
48	14:37:57	USE customer_sale_db	0 row(s) affected
49	14:37:57	SELECT DISTINCT c.* FROM Customer c JOIN Sale s ON c.cust_id = s.cust_id JOIN Item i ON s.item_id = i.it...	7 row(s) returned

6. Give a count of how many products have been bought by each customer.

```
USE customer_sale_db;
SELECT
    c.cust_name,
    SUM(s.qty_sold) AS total_products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_name	total_products
Vanshika Dubey	7
Riya	1
Karan	3
Sneha	1
Rahul	2
Priya	1
Arjun	4
Neha	2
Vikas	1

#	Time	Action	Message
50	14:39:27	USE customer_sale_db	0 row(s) affected
51	14:39:27	SELECT c.cust_name, SUM(s.qty_sold) AS total_products FROM Customer c JOIN Sale s ON c.cust_id ...	9 row(s) returned

7. Give a list of products bought by a customer having cust_id as 5.

```
USE customer_sale_db;
SELECT i.item_name, s.qty_sold
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

Result Grid | Filter Rows:

item_name	qty_sold
Watch	2

Output

Action Output

#	Time	Action	Message
54	14:42:45	USE customer_sale_db	0 row(s) affected
55	14:42:45	SELECT i.item_name, s.qty_sold FROM Sale s JOIN Item i ON s.item_id = i.item_id WHERE s.cust_id = 5 LIMI...	1 row(s) returned

8. List the item details which are sold as of today.

```
USE customer_sale_db;
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows:

item_id	item_name	price
101	Pen	10
102	Notebook	50
103	Bag	500
104	Bottle	150
105	Shoes	1200
106	Watch	2000

#	Time	Action	Message
56	14:47:42	USE customer_sale_db	0 row(s) affected
57	14:47:42	SELECT DISTINCT i.* FROM Item i JOIN Sale s ON i.item_id = s.item_id WHERE s.bill_date = CURDATE() LI...	6 row(s) returned

]

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount

```sql
USE customer_sale_db;
CREATE VIEW Bill_Details AS
SELECT
    s.bill_no,
    s.bill_date,
    s.cust_id,
    s.item_id,
    i.price,
    s.qty_sold,
    (i.price * s.qty_sold) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

```sql
SELECT * FROM Bill_Details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1	2026-02-28	1	101	10	5	50
1	2026-02-28	1	102	50	2	100
2	2026-02-28	2	103	500	1	500
3	2026-02-28	3	104	150	3	450
4	2026-02-28	4	105	1200	1	1200
5	2026-02-28	5	106	2000	2	4000
6	2026-02-27	6	107	350	1	350
7	2026-02-26	7	108	300	4	1200
8	2026-02-25	8	109	600	2	1200
9	2026-02-24	9	110	800	1	800

58	14:49:08	USE customer_sale_db	0 row(s) affected
59	14:49:08	CREATE VIEW Bill_Details AS SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id, i.price, s.qty_...	0 row(s) affected
60	14:49:49	USE customer_sale_db	0 row(s) affected
61	14:49:49	SELECT * FROM Bill_Details LIMIT 0, 1000	10 row(s) returned

10. Create a view which lists the daily sales date wise for the last one week.

```sql
USE customer_sale_db;
CREATE VIEW Weekly_Sales AS
SELECT
    bill_date,
    SUM(i.price * s.qty_sold) AS daily_total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY bill_date
ORDER BY bill_date;
SELECT * FROM Weekly_Sales;
```

Result Grid | Filter Rows:

bill_date	daily_total_sales
2026-02-24	800
2026-02-25	1200
2026-02-26	1200
2026-02-27	350
2026-02-28	6300

#	Time	Action	Message
62	14:52:04	USE customer_sale_db	0 row(s) affected
63	14:52:04	CREATE VIEW Weekly_Sales AS SELECT bill_date, SUM(i.price * s.qty_sold) AS daily_total_sales FRO...	0 row(s) affected
64	14:52:04	SELECT * FROM Weekly_Sales LIMIT 0, 1000	5 row(s) returned